

Kmp

前缀函数与 KMP 算法

字符串前缀和后缀定义

关于字符串前缀、真前缀，后缀、真后缀的定义详见 [字符串基础](#)

前缀函数

定义

给定一个长度为 n 的字符串，其 **前缀函数** 被定义为一个长度为 n 的数组。其中 l 的定义是：

1. 如果子串 $s[0:l]$ 有一对相等的真前缀与真后缀： $s[0:k]$ 和 $s[l-k:l]$ ，那么 k 就是这个相等的真前缀（或者真后缀，因为它们相等）的长度，也就是 l ；
2. 如果不止有一对相等的，那么 l 就是其中最长的那一对的长度；
3. 如果没有相等的，那么 $l = 0$ 。

简单来说 l 就是，子串 $s[0:l]$ 最长的相等的真前缀与真后缀的长度。

用数学语言描述如下：

特别地，规定 $l[0] = 0$ 。

过程

举例来说，对于字符串 `abcbabcd`，

$l[0] = 0$ ，因为 `a` 没有真前缀和真后缀，根据规定为 0

$l[1] = 0$ ，因为 `ab` 无相等的真前缀和真后缀

$l[2] = 0$ ，因为 `abc` 无相等的真前缀和真后缀

$l[3] = 1$ ，因为 `abca` 只有一对相等的真前缀和真后缀：`a`，长度为 1

$l[4] = 2$ ，因为 `abcab` 相等的真前缀和真后缀只有 `ab`，长度为 2

$l[5] = 3$ ，因为 `abcbabc` 相等的真前缀和真后缀只有 `abc`，长度为 3

$l[6] = 0$ ，因为 `abcbabcd` 无相等的真前缀和真后缀

同理可以计算字符串 `aabaaab` 的前缀函数为 `[0, 1, 2, 3, 4, 0, 0]`。

计算前缀函数的朴素算法

过程

一个直接按照定义计算前缀函数的算法流程：

- 在一个循环中以 i 的顺序计算前缀函数 $l[i]$ 的值（被赋值为 0 ）。

- 为了计算当前的前缀函数值，我们令变量 j 从最大的真前缀长度开始尝试。
- 如果当前长度下真前缀和真后缀相等，则此时长度为 j ，否则令 j 自减 1，继续匹配，直到 $j = 0$ 。
- 如果 $j = 0$ 并且仍没有任何一次匹配，则置 $pi[i] = 0$ 并移至下一个下标。

▼ 实现

具体实现如下：



C++PythonJava

```
1 // 注:
2 // string substr (size_t pos = 0, size_t len = npos) const;
3 vector<int> prefix_function(string s) {
4     int n = (int)s.length();
5     vector<int> pi(n);
6     for (int i = 1; i < n; i++)
7         for (int j = i; j >= 0; j--)
8             if (s.substr(0, j) == s.substr(i - j + 1, j)) {
9                 pi[i] = j;
10                break;
11            }
12     return pi;
13 }
```

```
1 def prefix_function(s):
2     n = len(s)
3     pi = [0] * n
4     for i in range(1, n):
5         for j in range(i, -1, -1):
6             if s[0:j] == s[i - j + 1 : i + 1]:
7                 pi[i] = j
8                 break
9     return pi
```

```
1 static int[] prefix_function(String s) {
2     int n = s.length();
3     int[] pi = new int[n];
4     for (int i = 1; i < n; i++) {
5         for (int j = i; j >= 0; j--) {
6             if (s.substring(0, j).equals(s.substring(i - j + 1, i + 1))) {
7                 pi[i] = j;
8                 break;
9             }
10        }
11    }
12    return pi;
13 }
```

显见该算法的时间复杂度为 $O(n^2)$ ，具有很大的改进空间。

计算前缀函数的高效算法

第一个优化

第一个重要的观察是 相邻的前缀函数值至多增加 1。

参照下图所示，只需如此考虑：当取一个尽可能大的 j 时，必然要求新增的 j 也与之对应的字符匹配，即，此时 $pi[i] = pi[i-1] + 1$ 。

所以当移动到下一个位置时，前缀函数的值要么增加一，要么维持不变，要么减少。

▼ 实现

此时的改进的算法为：



C++PythonJava

```
1 vector<int> prefix_function(string s) {
2     int n = (int)s.length();
3     vector<int> pi(n);
4     for (int i = 1; i < n; i++)
5         for (int j = pi[i - 1] + 1; j >= 0; j--) // improved: j=i => j=pi[i-1]+1
6             if (s.substr(0, j) == s.substr(i - j + 1, j)) {
7                 pi[i] = j;
8                 break;
9             }
10    return pi;
11 }
```

```
1 def prefix_function(s):
2     n = len(s)
3     pi = [0] * n
4     for i in range(1, n):
5         for j in range(pi[i - 1] + 1, -1, -1):
6             if s[0:j] == s[i - j + 1 : i + 1]:
7                 pi[i] = j
8                 break
9     return pi
```

```
1 static int[] prefix_function(String s) {
2     int n = s.length();
3     int[] pi = new int[n];
4     for (int i = 1; i < n; i++) {
5         for (int j = pi[i - 1] + 1; j >= 0; j--) {
6             if (s.substring(0, j).equals(s.substring(i - j + 1, i + 1))) {
7                 pi[i] = j;
8                 break;
9             }
10        }
11    }
12    return pi;
13 }
```

在这个初步改进的算法中，在计算每个 i 时，最好的情况是第一次字符串比较就完成了匹配，也就是说基础的字符串比较次数是 $n-1$ 次。

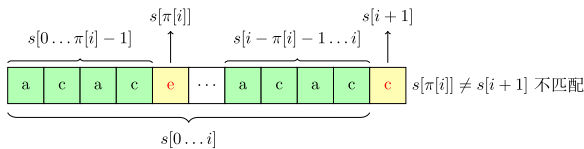
而由于存在 $j = pi[i-1]+1$ ($pi[0]=0$) 对于最大字符串比较次数的限制，可以看出每次只有在最好情况才会为字符串比较次数的上限积累 1，而每次超过一次的字符串比较消耗的是之后次数的增长空间。

由此我们可以得出字符串比较次数最多的一种情况：至少 1 次字符串比较次数的消耗和最多 $n-2$ 次比较次数的积累，此时字符串比较次数为 $n-1 + n-2 = 2n-3$ 。

可见经过此次优化，计算前缀函数只需要进行 n 次字符串比较，总复杂度降为了 $O(n)$ 。

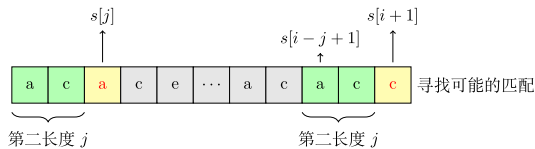
第二个优化

在第一个优化中，我们讨论了计算 $pi[i]$ 时的最好情况：，此时 $pi[i] = pi[i-1] + 1$ 。现在让我们沿着这个思路走得更远一点：讨论当 $pi[i] = 0$ 时如何跳转。



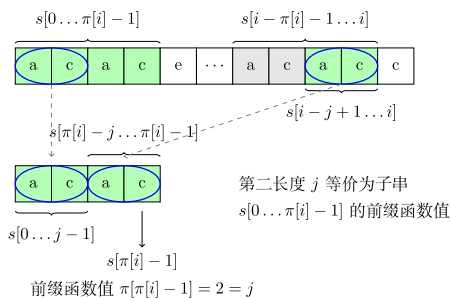
如上图所示，失配时，我们希望找到对于子串，仅次于 的第二长度，使得在位置 的前缀性质仍得以保持，也即：

如果我们找到了这样的长度，那么仅需要再次比较 和 。如果它们相等，那么就有 。否则，我们需要找到子串 仅次于 的第二长度，使得前缀性质得以保持，如此反复，直到 。如果 ，则 。第二次比较的示意图如下所示



观察上图可以发现，因为 ，所以对于 的第二长度，有这样的性质：

该公式的示意图如下所示：



也就是说 等价于子串 的前缀函数值，对应于上图下半部分，即 。同理，次于 的第二长度等价于 的前缀函数值，。

显然我们可以得到一个关于 的状态转移方程：

最终算法

所以最终我们可以构建一个不需要进行任何字符串比较，并且只进行 次操作的算法。

而且该算法的实现出人意外的短且直观：

▼ 实现



C++PythonJava

```

1 vector<int> prefix_function(string s) {
2     int n = (int)s.length();
3     vector<int> pi(n);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i - 1];
6         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
7         if (s[i] == s[j]) j++;
8         pi[i] = j;
9     }
10    return pi;
11 }

1 def prefix_function(s):
2     n = len(s)
3     pi = [0] * n
4     for i in range(1, n):
5         j = pi[i - 1]
6         while j > 0 and s[i] != s[j]:
7             j = pi[j - 1]
8         if s[i] == s[j]:
9             j += 1
10        pi[i] = j
11    return pi

1 static int[] prefix_function(String s) {
2     int n = s.length();
3     int[] pi = new int[n];
4     for (int i = 1; i < n; i++) {
5         int j = pi[i - 1];
6         while (j > 0 && s.charAt(i) != s.charAt(j)) {
7             j = pi[j - 1];
8         }
9         if (s.charAt(i) == s.charAt(j)) {
10            j++;
11        }
12        pi[i] = j;
13    }
14    return pi;
15 }

```

这是一个 **在线** 算法，即其当数据到达时处理它——举例来说，你可以一个字符一个字符的读取字符串，立即处理它们以计算出每个字符的前缀函数值。该算法仍然需要存储字符串本身以及先前计算过的前缀函数值，但如果我们已经预先知道该字符串前缀函数的最大可能取值，那么我们仅需要存储该字符串的前 个字符以及对应的前缀函数值。

应用

在字符串中查找子串：Knuth–Morris–Pratt 算法

该算法由 Knuth、Pratt 和 Morris 在 1977 年共同发布^[1]。

该任务是前缀函数的一个典型应用。

过程

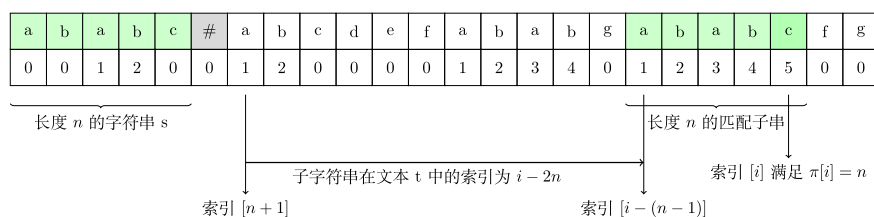
给定一个文本 和一个字符串，我们尝试找到并展示 在 中的所有出现（occurrence）。

为了简便起见，我们用 表示字符串 的长度，用 表示文本 的长度。

我们构造一个字符串，其中 为一个既不出现在 中也不出现在 中的分隔符。接下来计算该字符串的前缀函数。现在考

考虑该前缀函数除去最开始一个值（即属于字符串 s 和分隔符的函数值）后其余函数值的意义。根据定义， $\pi[i]$ 为右端点在 s 且同时为一个前缀的最长真子串的长度，具体到我们的这种情况下，其值为与 s 的前缀相同且右端点位于 s 的最长子串的长度。由于分隔符的存在，该长度不可能超过 n 。而如果等式 $\pi[i] = n$ 成立，则意味着 s 完整出现在该位置（即其右端点位于位置 i ）。注意该位置的下标是对字符串 t 而言的。

因此如果在某一位置 i 有 $\pi[i] = n$ 成立，则字符串 s 在字符串 t 的 i 处出现。下图所示为索引的示意图。



正如在前缀函数的计算中已经提到的那样，如果我们知道前缀函数的值永远不超过一特定值，那么我们不需要存储整个字符串以及整个前缀函数，而只需要二者开头的一部分。在我们这种情况下这意味着只需要存储字符串 s 以及相应的前缀函数值即可。我们可以一次读入字符串 t 的一个字符并计算当前位置的前缀函数值。

因此 Knuth–Morris–Pratt 算法（简称 KMP 算法）用 $O(n)$ 的时间以及 $O(n)$ 的内存解决了该问题。

▼ 实现



C++PythonJava

```
1 vector<int> find_occurrences(string text, string pattern) {
2     string cur = pattern + '#' + text;
3     int sz1 = text.size(), sz2 = pattern.size();
4     vector<int> v;
5     vector<int> lps = prefix_function(cur);
6     for (int i = sz2 + 1; i <= sz1 + sz2; i++) {
7         if (lps[i] == sz2) v.push_back(i - 2 * sz2);
8     }
9     return v;
10 }
```

```
1 def find_occurrences(t, s):
2     cur = s + "#" + t
3     sz1, sz2 = len(t), len(s)
4     ret = []
5     lps = prefix_function(cur)
6     for i in range(sz2 + 1, sz1 + sz2 + 1):
7         if lps[i] == sz2:
8             ret.append(i - 2 * sz2)
9     return ret
```

```
1 static List<Integer> find_occurrences(String text, String pattern) {
2     String cur = pattern + '#' + text;
3     int sz1 = text.length(), sz2 = pattern.length();
4     List<Integer> v = new ArrayList<>();
5     int[] lps = prefix_function(cur);
6     for (int i = sz2 + 1; i <= sz1 + sz2; i++) {
7         if (lps[i] == sz2) {
8             v.add(i - 2 * sz2);
9         }
10    }
11    return v;
12 }
```

字符串的周期

对字符串 s 和 t ，若 s 对所有 i 成立，则称 s 是 t 的周期。

对字符串 s 和 t ，若 s 长度为 m 的前缀和长度为 n 的后缀相等，就称 s 长度为 m 的前缀是 t 的 border。

由 s 有长度为 m 的 border 可以推导出 s 是 t 的周期。

根据前缀函数的定义，可以得到 s 所有的 border 长度，即 [2](#)。

所以根据前缀函数可以在 $O(n)$ 的时间内计算出 s 所有的周期。其中，由于 s 是 s 最长 border 的长度，所以 s 是 s 的最小周期。

统计每个前缀的出现次数

在该节我们将同时讨论两个问题。给定一个长度为 n 的字符串 s ，在问题的第一个变种中我们希望统计每个前缀 $s[0..i]$ 在同一个字符串的出现次数，在问题的第二个变种中我们希望统计每个前缀 $s[0..i]$ 在另一个给定字符串 t 中的出现次数。

首先让我们来解决第一个问题。考虑位置 i 的前缀函数值 $pi[i]$ 。根据定义，其意味着字符串 s 一个长度为 $pi[i]$ 的前缀在位置 i 出现并以 i 为右端点，同时不存在一个更长的前缀满足前述定义。与此同时，更短的前缀可能以该位置为右端点。容易看出，我们遇到了在计算前缀函数时已经回答过的问题：给定一个长度为 $pi[i]$ 的前缀，同时其也是一个右端点位于 i 的后缀，下一个更小的前缀长度是多少？该长度的前缀需同时也是一个右端点为 i 的后缀。因此以位置 i 为右端点，有长度为 $pi[i]$ 的前缀，有长度为 $pi[pi[i]]$ 的前缀，有长度为 $pi[pi[pi[i]]]$ 的前缀，等等，直到长度变为 0 。故而我们可以通过下述方式计算答案。

▼ 实现



C++Python

```
1 vector<int> ans(n + 1);
2 for (int i = 0; i < n; i++) ans[pi[i]]++;
3 for (int i = n - 1; i > 0; i--) ans[pi[i - 1]] += ans[i];
4 for (int i = 0; i <= n; i++) ans[i]++;
```

```
1 ans = [0] * (n + 1)
2 for i in range(0, n):
3     ans[pi[i]] += 1
4 for i in range(n - 1, 0, -1):
5     ans[pi[i - 1]] += ans[i]
6 for i in range(0, n + 1):
7     ans[i] += 1
```

解释

在上述代码中我们首先统计每个前缀函数值在数组 ans 中出现了多少次，然后再计算最后答案：如果我们知道长度为 i 的前缀出现了恰好 $ans[i]$ 次，那么该值必须被叠加至其最长的既是后缀也是前缀的子串的出现次数中。在最后，为了统计原始的前缀，我们对每个结果加 1 。

现在考虑第二个问题。我们应用来自 Knuth–Morris–Pratt 的技巧：构造一个字符串 s 并计算其前缀函数。与第一个问题唯一的不同之处在于，我们只关心与字符串 s 相关的前缀函数值，即 pi 的 $0..n$ 。有了这些值之后，我们可以同样应用在第一个问题中的算法来解决该问题。

一个字符串中本质不同子串的数目

给定一个长度为 n 的字符串 s ，我们希望计算其本质不同子串的数目。

我们将迭代的解决该问题。换句话说，在知道了当前的本质不同子串的数目的情况下，我们要找出一种在 s 末尾添加一个字符后重新计算该数目的方法。

令 ans 为当前的本质不同子串数量。我们添加一个新的字符 c 至 s 。显然，会有一些新的子串以字符 c 结尾。我们希望对这些以该字符结尾且我们之前未曾遇到的子串计数。

构造字符串 并将其反转得到字符串 。现在我们的任务变为计算有多少 的前缀未在 的其余任何地方出现。如果我们计算了 的前缀函数最大值，那么最长的出现在 中的前缀其长度为 。自然的，所有更短的前缀也出现了。

因此，当添加了一个新字符后新出现的子串数目为 。

所以对于每个添加的字符，我们可以在 的时间内计算新子串的数目，故最终复杂度为 。

值得注意的是，我们也可以重新计算在头部添加一个字符，或者从尾或者头移除一个字符时的本质不同子串数目。

字符串压缩

给定一个长度为 的字符串，我们希望找到其最短的「压缩」表示，也即我们希望寻找一个最短的字符串，使得 可以被 的一份或多份拷贝的拼接表示。

显然，我们只需要找到 的长度即可。知道了该长度，该问题的答案即为长度为该值的 的前缀。

让我们计算 的前缀函数。通过使用该函数的最后一个值，我们定义值 。我们将证明，如果 整除，那么 就是答案，否则不存在一个有效的压缩，故答案为 。

假定 可被 整除。那么字符串可被划分为长度为 的若干块。根据前缀函数的定义，该字符串长度为 的前缀等于其后缀。但是这意味着最后一个块同倒数第二个块相等，并且倒数第二个块同倒数第三个块相等，等等。作为其结果，所有块都是相等的，因此我们可以将字符串 压缩至长度 。

▼ 证明

诚然，我们仍需证明该值为最优解。实际上，如果有一个比 更小的压缩表示，那么前缀函数的最后一个值 必定比 要大。因此 就是答案。

现在假设 不可以被 整除，我们将通过反证法证明这意味着答案为 [1](#)。假设其最小压缩表示 的长度为 （整除），字符串 被划分为 块。那么前缀函数的最后一个值 必定大于 （如果等于则 可被 整除），也即其所表示的后缀将部分的覆盖第一个块。现在考虑字符串的第二个块。该块有两种解释：第一种为，另一种为。由于两种解释对应同一个字符串，因此可得到 个方程组成的方程组，该方程组可简写为，其中 表示模 意义下的最小非负剩余。

根据扩展欧几里得算法我们可以得到一组 和 使得 。通过与等式 适当叠加我们可以得到一组 和 使得 。这意味着通过不断应用前述方程组中的方程我们可以得到新的方程组 。

由于 整除，这意味着 是 的一个周期。又因为，故有，所以 是一个比 更小的 的周期。因此字符串 有一个长度为 的压缩表示，同 的最小性矛盾。

综上所述，不存在一个长度小于 的压缩表示，因此答案为 。

根据前缀函数构建一个自动机

让我们重新回到通过一个分隔符将两个字符串拼接的新字符串。对于字符串 和 我们计算 的前缀函数。显然，因为 是一个分隔符，前缀函数值永远不会超过 。因此我们只需要存储字符串 和其对应的前缀函数值，之后就可以动态计算对于之后所有字符的前缀函数值：

实际上在这种情况下，知道 的下一个字符 以及之前位置的前缀函数值便足以计算下一个位置的前缀函数值，而不需要用到任何其它 的字符和对应的前缀函数值。

换句话说，我们可以构造一个 **自动机**（一个有限状态机）：其状态为当前的前缀函数值，而从一个状态到另一个状态的转移则由下一个字符确定。

因此，即使没有字符串，我们同样可以应用构造转移表的算法构造一个转移表：

▼ 实现

```
1 void compute_automaton(string s, vector<vector<int>>& aut) {
2     s += '#';
3     int n = s.size();
4     vector<int> pi = prefix_function(s);
5     aut.assign(n, vector<int>(26));
6     for (int i = 0; i < n; i++) {
7         for (int c = 0; c < 26; c++) {
8             int j = i;
9             while (j > 0 && 'a' + c != s[j]) j = pi[j - 1];
10            if ('a' + c == s[j]) j++;
11            aut[i][c] = j;
12        }
13    }
14 }
```

然而在这种形式下，对于小写字母表，算法的时间复杂度为 $O(n^2)$ 。注意到我们可以应用动态规划来利用表中已计算过的部分。只要我们从值 i 变化到 $i+1$ ，那么我们实际上在说转移 i 所到达的状态同转移 $i-1$ 一样，但该答案我们之前已经精确计算过了。

▼ 实现

```
1 void compute_automaton(string s, vector<vector<int>>& aut) {
2     s += '#';
3     int n = s.size();
4     vector<int> pi = prefix_function(s);
5     aut.assign(n, vector<int>(26));
6     for (int i = 0; i < n; i++) {
7         for (int c = 0; c < 26; c++) {
8             if (i > 0 && 'a' + c != s[i])
9                 aut[i][c] = aut[pi[i - 1]][c];
10            else
11                aut[i][c] = i + ('a' + c == s[i]);
12        }
13    }
14 }
```

最终我们可在 $O(n)$ 的时间复杂度内构造该自动机。

该自动机在什么时候有用呢？首先，记得大部分时候我们为了一个目的使用字符串的前缀函数：寻找字符串 s 在字符串 t 中的所有出现。

因此使用该自动机的最直接的好处是 **加速计算字符串 s 的前缀函数**。

通过构建 A 的自动机，我们不再需要存储字符串 s 以及其对应的前缀函数值。所有转移已经在表中计算过了。

但除此以外，还有第二个不那么直接的应用。我们可以在字符串 s 是 **某些通过一些规则构造的巨型字符串** 时，使用该自动机加速计算。Gray 字符串，或者一个由一些短的输入串的递归组合所构造的字符串都是这种例子。

出于完整性考虑，我们来解决这样一个问题：给定一个数 n ，以及一个长度 m 的字符串 t ，我们需要计算 n 在第 i 个 Gray 字符串中的出现次数。回想起 Gray 字符串以下述方式定义：

由于其天文数字般的长度，在这种情况下即使构造字符串 G_n 都是不可能的：第 n 个 Gray 字符串有 2^n 个字符。然而我们可以在仅仅知道开头若干前缀函数值的情况下，有效计算该字符串末尾的前缀函数值。

除了自动机之外，我们同时需要计算值 $f(i, c)$ ：在从状态 i 开始处理 c 后的自动机的状态，以及值 $g(i, c)$ ：当从状态 i 开始处理 c 后，

在 中的出现次数。实际上 为在执行操作时前缀函数取值为 的次数。易得问题的答案为 。

我们该如何计算这些值呢？首先根据定义，初始条件为 以及 。之后所有值可以通过先前的值以及使用自动机计算得到。为了对某个 计算相应值，回想起字符串 由 ，字母表中第 个字符，以及 三者拼接而成。因此自动机会途径下列状态：

的值同样可被简单计算。

其中 当其中表达式取值为真时值为 ，否则为 。综上，我们已经可以解决关于 Gray 字符串的问题，以及一大类与之类似的问题。举例来说，应用同样的方法可以解决下列问题：给定一个字符串 以及一些模式 ，其中每个模式以下列方式给出：该模式由普通字符组成，当中可能以 的形式递归插入先前的字符串，也即在该位置我们必须插入字符串 次。以下是这些模式的一个例子：

递归代入会使字符串长度爆炸式增长，他们的长度甚至可以达到 的数量级。而我们必须找到字符串 在每个字符串中的出现次数。

该问题同样可通过构造前缀函数的自动机解决。同之前一样，我们利用先前计算过的结果对每个模式计算其转移然后相应统计答案即可。

练习题目

- [UVa 455 "Periodic Strings"](#)
- [UVa 11022 "String Factoring"](#)
- [UVa 11452 "Dancing the Cheeky-Cheeky"](#)
- [UVa 12604 - Caesar Cipher](#)
- [UVa 12467 - Secret Word](#)
- [UVa 11019 - Matrix Matcher](#)
- [SPOJ - Pattern Find](#)
- [Codeforces - Anthem of Berland](#)
- [Codeforces - MUH and Cube Walls](#)

参考资料与注释

本页面主要译自博文 [Префикс-функция. Алгоритм Кнута-Морриса-Пратта](#) 与其英文翻译版 [Prefix function. Knuth–Morris–Pratt algorithm](#)。其中俄文版版权协议为 Public Domain + Leave a Link；英文版版权协议为 CC-BY-SA 4.0。

-
1. 在俄文版及英文版中该部分证明均疑似有误。本文章中的该部分证明由作者自行添加。 [↩](#)
 2. [金策 - 字符串算法选讲](#) [↩](#)
-

本页面最近更新：2025/4/6 12:15:33, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[StudyingFather](#), [Back1ght](#), [greyqz](#), [iamtwz](#), [lr1d](#), [LeoJacob](#), [minghu6](#), [Xeonacid](#), [akakw1](#), [bitsstdcheee](#), [CCXXI](#), [countercurrent-time](#), [Enter-tainer](#), [gi-b716](#), [Great-designer](#), [hsiachi](#), [Jvaeyhcd](#), [ksyx](#),

[lotato](#), [Marcythm](#), [MarkJoson](#), [Menci](#), [NachtgeistW](#), [nexplorer-3e](#), [shawllewyw](#), [silverling](#), [SukkaW](#), [Tiphereth-A](#), [TrisolarisHD](#), [WException](#), [xiaocao666tzh](#), [xzcxcyy](#), [zychen20](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用